Informazione e sua codifica

PFI 2026

Prof. Giorgio Gambosi

Università di Roma "Tor Vergata"

Abstract. *Queste note introducono il concetto di informazione e ne illustrano la formalizzazione matematica attraverso il collegamento con la probabilità e l'incertezza. Si parte da una riflessione sul ruolo dell'informatica nella rappresentazione e nella gestione dell'informazione, si introduce la nozione di bit come unità di misura, si definisce la funzione di informazione di Shannon e si arriva al concetto di entropia come misura dell'incertezza media di una variabile casuale. I concetti vengono illustrati attraverso esempi concreti, tra cui il celebre problema delle dodici palline, il problema dell'indovina-numero e l'ordinamento per confronti. Si affronta poi l'analisi statistica del linguaggio naturale, con un approfondimento sui moderni modelli del linguaggio basati su reti neurali. La seconda parte introduce il modello sorgente-canale-destinatario e sviluppa la teoria della codifica: codifica di sorgente (Shannon-Fano, Huffman) e codifica di canale (distanza di Hamming, codice di Hamming* $(7, 4)$*), mostrando come compressione e correzione degli errori siano due facce complementari della stessa teoria.*

a.a. 2025–2026

Indice

1. Informatica, informazione e automatismo

L'informatica può essere descritta come la disciplina che si occupa della **risoluzione automatica di problemi**. Ma che cosa vuol dire, esattamente, che un problema viene risolto in modo automatico?

Per rispondere a questa domanda è utile distinguere due grandi filoni di attività che caratterizzano la disciplina. Il primo riguarda la *risoluzione di problemi* in senso stretto: dato un problema, ci chiediamo se esiste un metodo per risolverlo, e se sì, quanto tempo e quante risorse computazionali sono necessarie. Il secondo riguarda invece la *rappresentazione e gestione dell'informazione*: come possiamo descrivere l'informazione in modo preciso? Come possiamo rappresentarla in modo efficiente, trasformarla, derivare nuova informazione da quella disponibile, e comunicarla in modo non ambiguo?

In entrambi i casi, la parola chiave è **automatico**. Questo termine evoca l'immagine di un *agente* capace di comprendere e svolgere alcune operazioni di base, al quale si vuole comunicare come effettuare azioni più complesse, a partire da ciò che sa già fare. L'attività dell'informatico consiste dunque in due passi: trovare un modo efficiente per risolvere un problema, e poi descrivere quel modo in maniera *esauriente* e *non ambigua*, in modo tale che l'agente possa eseguirlo senza dover interpretare né colmare lacune.

Il caso più familiare di agente automatico è, naturalmente, il computer. Ma è importante non confondere il caso generale con il caso particolare: il computer è soltanto un agente particolarmente veloce nell'eseguire le istruzioni che gli vengono fornite. La vera sfida intellettuale non sta nella macchina, ma nel modo in cui si concepisce e si comunica la soluzione.

1.1. Matematizzare l'informazione

Per poter elaborare l'informazione in modo automatico, è necessario trattarla con rigore: dobbiamo in qualche modo **matematizzarla**, renderla misurabile come una qualunque grandezza fisica. Non ci basta un concetto intuitivo e vago di "informazione"; abbiamo bisogno di qualcosa che si possa quantificare, confrontare, sommare.

Come vedremo, la chiave per farlo sta nel collegare l'informazione all'**incertezza** e, attraverso di essa, alla **probabilità**. L'idea di fondo è semplice quanto profonda: ricevere un'informazione significa ridurre la nostra incertezza su qualcosa. Maggiore era la nostra incertezza prima di ricevere la comunicazione, maggiore è l'informazione che la comunicazione stessa ci ha trasmesso.

2. Un puzzle per cominciare

Prima di affrontare la teoria in modo sistematico, è utile partire da un problema concreto che illustri in modo vivido il legame tra informazione e incertezza.

2.1. Il problema delle dodici palline

Immaginiamo di avere dodici palline apparentemente identiche. Undici di esse hanno esattamente lo stesso peso; una sola ha un peso leggermente diverso, ma non sappiamo se sia più pesante o più leggera delle altre. Abbiamo a disposizione una

bilancia a due piatti: su ciascun piatto possiamo disporre un qualunque sottoinsieme delle palline, e la bilancia ci dice soltanto se i due gruppi hanno lo stesso peso oppure, in caso contrario, quale dei due sia più pesante.

Ciascuna pesata ha quindi esattamente tre possibili esiti:

(1) i due gruppi hanno lo stesso peso;

(2) il primo gruppo è più pesante;

(3) il secondo gruppo è più pesante.

Esempio 2.1— Il problema delle palline

Vogliamo individuare la pallina diversa e determinare se è più pesante o più leggera rispetto alle altre. Il *minimo numero di pesate* necessario a risolvere sempre il problema — qualunque sia la pallina anomala e qualunque sia il suo tipo di diversità — è la domanda centrale del puzzle.

2.2. L'incertezza come punto di partenza

Una sequenza di pesate è un processo che riduce progressivamente la nostra incertezza. Nella situazione iniziale, la pallina diversa potrebbe essere una qualunque delle dodici, e potrebbe essere sia più pesante sia più leggera: in totale, ci sono $12 \times 2 = 24$ possibilità distinte tutte equiprobabili. Alla fine del processo, siamo arrivati a una sola possibilità, e la nostra incertezza è azzerata.

Vale la pena di soffermarsi su questo punto. Non si tratta soltanto di trovare la pallina giusta: si tratta di portare la nostra *conoscenza* da una situazione di massima incertezza (24 casi) a una di certezza assoluta (un solo caso). Ogni pesata è dunque un atto di acquisizione di informazione, e il puzzle si propone di massimizzare l'efficienza di questo processo.

2.3. Questioni aperte

Il problema delle dodici palline solleva una serie di domande naturali e profonde, che costituiscono il filo conduttore delle sezioni seguenti:

- Come si misura la quantità di informazione che ricaviamo da una pesata? In modo simmetrico, come si misura l'incertezza che ancora rimane dopo un certo numero di pesate?
- Quanta informazione complessiva dobbiamo acquisire per risolvere il puzzle? È possibile, cioè, stabilire a priori quante pesate saranno necessarie?
- Quanta informazione si ottiene, ad esempio, confrontando due gruppi di sei palline? E quanta se si confrontano due gruppi di quattro, lasciandone quattro da parte?
- Qual è la strategia di pesata che massimizza, ad ogni passo, la quantità di informazione acquisita?

Rispondere a queste domande richiede di sviluppare una teoria quantitativa dell'informazione, che è esattamente ciò di cui ci occuperemo nelle sezioni seguenti. Ritorneremo al puzzle dopo aver sviluppato gli strumenti necessari.

3. Misurare l'informazione: il bit

Trattiamo l'informazione come una grandezza fisica, esattamente come la lunghezza si misura in metri e il tempo si misura in secondi. L'unità di misura dell'informazione è il **bit**.

Osservazione 3.1— Quantità vs. contenuto

Conoscere la quantità di informazione non è la stessa cosa che conoscere l'informazione stessa. Sapere quanti bit di informazione ci dà l'individuazione della pallina diversa è ben diverso dal sapere *quale* pallina sia quella diversa. La prima è una misura astratta; la seconda è il contenuto concreto dell'informazione.

3.1. Da casi possibili a bit

Consideriamo un esempio elementare. Supponiamo che Alice abbia osservato il cielo e voglia comunicare a Bruno come sarà il tempo domani. Gli esiti possibili sono soltanto due: *pioverà* oppure *bel tempo*. Alice non ha bisogno di scrivere una lunga lettera: le è sufficiente comunicare un singolo valore che discrimini tra le due possibilità, ad esempio 0 per il bel tempo e 1 per la pioggia. In questo caso, diciamo che l'informazione da comunicare è pari a **1 bit**.

La stessa situazione si presenta per la domanda "il mio amico viene al cinema?": anche qui le possibilità sono due (*sì*/*no*), e un solo bit è sufficiente per comunicare la risposta.

Se invece Alice volesse comunicare *sia* il meteo *sia* la risposta dell'amico, avrebbe bisogno di 2 bit: le combinazioni possibili sono quattro (*piove e sì*, *piove e no*, *bello e sì*, *bello e no*), e con 2 bit si possono codificare esattamente quattro valori distinti. In generale, se il numero di casi possibili raddoppia, il numero di bit aumenta di uno.

Esempio 3.1— Bit e casi possibili

Mettiamo in relazione il numero di bit b con il numero di casi possibili c:

c	2	4	8	16	32	64	$\cdots$
$b = \log_2 c$	1	2	3	4	5	6	$\cdots$

La legge che lega il numero di bit b al numero di casi c è $c = 2^b$, ovvero, invertendo, $b = \log_2 c$.

3.2. Casi non interi: quando c non è una potenza di 2

La formula $b = \log_2 c$ vale in generale, anche quando c non è una potenza di 2. In questo caso, il numero di bit risulta non intero. Per esempio, il lancio di un dado a sei facce ha $c = 6$ esiti possibili, per cui

$$b = \log_2 6 \simeq 2{,}585 \text{ bit}.$$

Se vogliamo effettivamente codificare l'esito con un numero intero di bit, dobbiamo prendere la *funzione soffitto*, indicata con $\lceil\cdot\rceil$: la notazione $\lceil x\rceil$ denota il più piccolo intero non inferiore a x. Per esempio, $\lceil 2{,}585\rceil = 3$, $\lceil 3\rceil = 3$, $\lceil 3{,}1\rceil = 4$. Nel caso del dado, $\lceil \log_2 6\rceil = 3$: con 3 bit si possono rappresentare $2^3 = 8$ valori distinti, dei quali solo 6 corrispondono alle facce del dado; le due sequenze di bit rimanenti non verrebbero mai usate.

Osservazione 3.2— Additività dell'informazione

Una proprietà fondamentale è che l'informazione è **additiva**: l'informazione associata a due eventi indipendenti è la somma delle informazioni dei singoli eventi. Questo discende direttamente dalla proprietà del logaritmo $\log_2(x\cdot y) = \log_2 x + \log_2 y$: se il primo evento ha c_1 esiti possibili e il secondo ne ha c_2, la coppia ha $c_1 \cdot c_2$ esiti possibili, e l'informazione totale è $\log_2(c_1c_2) = \log_2 c_1 + \log_2 c_2$.

Esempio 3.2— Dadi di Dungeons & Dragons

In alcuni giochi di ruolo, come Dungeons & Dragons, si utilizzano dadi con numeri di facce diversi: 4, 6, 8, 10, 12, 20. Per esempio, il lancio di un dado da 8 e uno da 10 ha $8 \times 10 = 80$ esiti possibili; l'informazione corrispondente è $\log_2 80 \simeq 6{,}32$ bit, pari alla somma $\log_2 8 + \log_2 10$. Quale coppia di dadi fornisce più informazione: un d20 e un d4, oppure tre d6? Nel primo caso: $\log_2 20+\log_2 4 \simeq 4{,}32+2 = 6{,}32$ bit. Nel secondo: $3\times\log_2 6 \simeq 3\times 2{,}585 = 7{,}76$ bit. Il lancio di tre d6 è dunque più informativo.

3.3. Conoscenza parziale e riduzione dell'incertezza

Una situazione frequente e istruttiva è quella in cui, anziché conoscere l'esito completo di un evento, apprendiamo solo alcune sue caratteristiche parziali. In questo caso, l'informazione ricevuta è esattamente la riduzione dell'incertezza: la differenza tra ciò che non sapevamo prima e ciò che non sappiamo ancora dopo.

Esempio 3.3— Informazione parziale con due dadi

Un amico lancia un dado da 10 e uno da 12, e ci comunica che nel risultato il primo dado non ha dato 5 e il secondo è pari.
Prima del lancio, la nostra incertezza sull'esito è $\log_2 10 + \log_2 12 \simeq 6{,}907$ bit.
Dopo la comunicazione parziale, i casi ancora possibili sono $(10-1)\times 6 = 54$: il primo dado può essere uno dei 9 valori diversi da 5, il secondo uno dei 6 valori pari $(2, 4, 6, 8, 10, 12)$. L'incertezza residua è $\log_2 54 \simeq 5{,}755$ bit. La riduzione dell'incertezza, ovvero l'informazione fornita dall'amico, è dunque di circa $6{,}907 - 5{,}755 \simeq 1{,}152$ bit.

Questo esempio illustra un principio generale: la quantità di informazione trasmessa da una comunicazione si misura come la differenza tra l'incertezza *prima* e l'incertezza *dopo* averla ricevuta. Ci si può chiedere quale tipo di comunicazione, tra più alternative, fornisca la maggiore informazione. La risposta dipende interamente dal numero di casi che ciascuna comunicazione è in grado di escludere.

Esempio 3.4— Quale comunicazione è più informativa?

Lo stesso amico lancia di nuovo un dado da 10 e uno da 12, e ci propone tre comunicazioni alternative:

1. "il lancio del primo dado ha dato valore pari";
2. "il lancio del secondo dado ha dato un valore divisibile per 3";
3. "il primo dado non ha dato 3 o 4 e il secondo non ha dato 7, 8 o 9".

Nel primo caso, la comunicazione ci dice quale tra le due possibilità (pari/dispari) si è verificata per il primo dado: ciò corrisponde a $\log_2 2 = 1$ bit.
Nel secondo caso, ci viene detto quale delle tre classi di resto (divisibile per 3, resto 1, resto 2) ha prodotto il secondo dado, fornendo $\log_2 3 \simeq 1{,}585$ bit.
Nel terzo caso, l'incertezza sul primo dado passa da $\log_2 10 \simeq 3{,}322$ a $\log_2 8 = 3$ bit (guadagno $\simeq 0{,}322$), e quella sul secondo da $\log_2 12 \simeq 3{,}585$ a $\log_2 9 \simeq 3{,}170$ bit (guadagno $\simeq 0{,}415$), per un totale di circa 0,737 bit.
La comunicazione più informativa è la seconda ($\simeq 1{,}585$ bit), seguita dalla prima (1 bit) e dalla terza ($\simeq 0{,}737$ bit). Ciò che conta è esclusivamente quanti casi ciascuna comunicazione esclude.

4. Informazione e probabilità

Fin qui abbiamo assunto che tutti gli esiti fossero *equiprobabili*. Ma nella realtà le cose sono spesso più complicate: alcuni esiti sono più probabili di altri, e questa asimmetria ha un impatto diretto sulla quantità di informazione che ci viene trasmessa.

4.1. Un breve ripasso di probabilità

Quando osserviamo due eventi X e Y, distinguiamo tra:

- la **probabilità marginale** $p(X)$, ovvero la probabilità che avvenga X indipendentemente da Y;
- la **probabilità congiunta** $p(X, Y)$, ovvero la probabilità che avvengano entrambi;
- la **probabilità condizionata** $p(X|Y)$, ovvero la probabilità che avvenga X *dato che* Y si è verificato.

Esempio 4.1— Verifica di matematica

In una classe di 25 studenti, i risultati di una verifica di matematica sono così distribuiti:

	Maschi (M)	Femmine (F)
Sufficiente	10	7
Insufficiente	6	2

La probabilità che uno studente scelto a caso sia femmina è $p(F) = 9/25$. La probabilità che un compito sufficiente appartenga a un maschio è $p(M \mid \text{Suff.}) = 10/17$. La probabilità congiunta che un compito sia sufficiente *e* appartenga a un maschio è $p(\text{Suff.}, M) = 10/25$.

4.2. L'informazione come riduzione dell'incertezza

Consideriamo una classe di 32 studenti: 30 maschi e 2 femmine. Vogliamo scegliere un rappresentante di classe tra tutti gli studenti. Quanta incertezza abbiamo su chi verrà scelto?

Con 32 possibilità equiprobabili, l'incertezza iniziale è $I(E) = \log_2 32 = 5$ bit. Ora supponiamo di sapere che il rappresentante è una femmina. Dato che le femmine sono soltanto 2, l'incertezza residua $I(E|F) = \log_2 2 = 1$ bit. L'informazione acquisita è quindi $I(E) - I(E|F) = 5 - 1 = 4$ bit.

Supponiamo invece di sapere che il rappresentante è un maschio. I maschi sono 30, per cui l'incertezza residua è $I(E|M) = \log_2 30 \simeq 4{,}907$ bit. L'informazione acquisita è soltanto $5 - 4{,}907 \simeq 0{,}093$ bit.

Questo risultato è sorprendente: sapere che il rappresentante è femmina ci fornisce circa 43 volte più informazione di sapere che è maschio! La ragione è semplice: le femmine sono pochissime, e saperlo è una grande *sorpresa*. I maschi, al contrario, sono la norma: sapere che il rappresentante è maschio non modifica quasi nulla la nostra situazione di partenza.

4.3. La funzione di informazione di Shannon

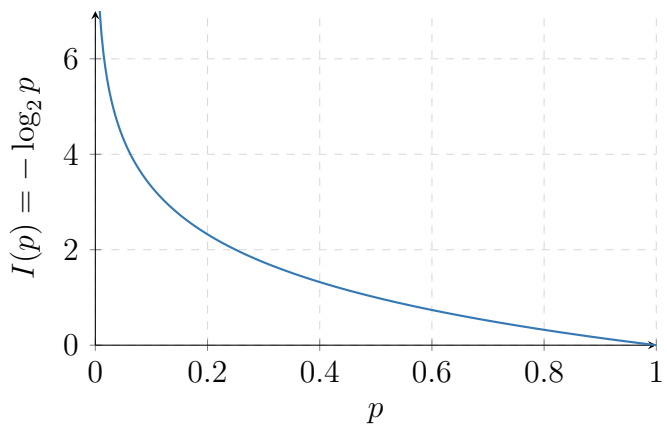
Il legame tra probabilità e informazione è formalizzato dalla **funzione di informazione di Shannon**.

Definizione 4.1— Funzione di informazione

La quantità di informazione associata a un evento di probabilità p è

$$I(p) = -\log_2 p = \log_2 \frac{1}{p}.$$

Si misura in bit. Più l'evento è raro (piccolo p), maggiore è l'informazione che ci trasmette quando si verifica.



La funzione $I(p)$ ha tre caratteristiche che vale la pena leggere esplicitamente sul grafico. Primo, vale esattamente 0 per $p = 1$: un evento certo non ci trasmette alcuna informazione, perché già sappiamo che avverrà. Secondo, cresce senza limite al diminuire di p: un evento rarissimo, se si verifica, ci trasmette una quantità arbitrariamente grande di informazione. Terzo, la curva è convessa e ripida vicino a $p = 0$, il che significa che la sorpresa cresce molto rapidamente al diminuire della probabilità.

Tornando all'esempio del rappresentante di classe, possiamo ora verificare che $I(E) = -\log_2(1/32) = 5$ bit, e che $I(E|M) = -\log_2(1/30) \simeq 4{,}907$ bit, in accordo con i calcoli precedenti.

Osservazione 4.1— Informazione e probabilità condizionata

L'informazione acquisita conoscendo un evento M di probabilità $p(M)$ è pari a $-\log_2 p(M)$. Questo è coerente con l'interpretazione dell'informazione come sorpresa: un evento molto probabile ci dice poco di nuovo; un evento inatteso ci costringe a rivedere profondamente le nostre aspettative.

Esempio 4.2— Informazione nella partita Roma–Milan

Supponiamo che le quote per Roma–Milan indichino le seguenti probabilità: vittoria della Roma al 50%, pareggio al 30%, vittoria del Milan al 20%. Le quantità di informazione associate ai tre esiti sono:

$$\begin{aligned} I(\text{Roma vince}) &= \log_2 \frac{100}{50} = \log_2 2 = 1 \text{ bit}, \\ I(\text{Pareggio}) &= \log_2 \frac{100}{30} \simeq 1{,}737 \text{ bit}, \\ I(\text{Milan vince}) &= \log_2 \frac{100}{20} = \log_2 5 \simeq 2{,}322 \text{ bit}. \end{aligned}$$

La vittoria del Milan, essendo l'esito meno atteso, è quella che ci fornisce la maggiore quantità di informazione.

4.4. L'identità algebrica fondamentale

È importante notare che la definizione di informazione come $-\log_2 p$ non è arbitraria: essa emerge in modo naturale dal calcolo della riduzione di incertezza. Riprendiamo l'esempio del rappresentante di classe. Per uno specifico studente maschio, la probabilità di essere scelto condizionata al fatto che il rappresentante sia femmina è $p(E|F) = 0$, e quindi $p(E) = p(E|M) \cdot p(M)$, da cui $p(M) = p(E)/p(E|M)$.

L'informazione acquisita sapendo che il rappresentante è maschio diventa allora:

$$I(E) - I(E|M) = -\log_2 p(E) - (-\log_2 p(E|M)) = -\log_2 \frac{p(E)}{p(E|M)} = -\log_2 p(M).$$

Analogamente, l'informazione acquisita sapendo che è femmina è $-\log_2 p(F)$.

Proposizione 4.1— Informazione di un evento come sorpresa

La quantità di informazione acquisita scoprendo che si è verificato un evento M di probabilità $p(M)$ è pari a $-\log_2 p(M)$. Questa formula è coerente con la definizione di Shannon: la sorpresa di un evento è inversamente proporzionale alla sua probabilità.

Un evento quasi certo (alta probabilità) non ci sorprende e non ci dice quasi nulla; un evento molto improbabile (bassa probabilità), quando accade, ci trasmette invece una grande quantità di informazione. Il termine **sorpresa** è pertanto un sinonimo di *informazione* nel vocabolario della teoria.

5. Entropia: misurare l'incertezza media

La funzione di informazione $I(p) = -\log_2 p$ ci dice quanta informazione ci trasmette un singolo esito quando si verifica. Ma prima che l'esito si conosca, non sappiamo quale si verificherà. Abbiamo bisogno di una misura della nostra incertezza *prima* dell'osservazione, che tenga conto di tutti gli esiti possibili e delle loro probabilità.

5.1. Perché $\log_2 n$ non basta

Nella Sezione 3 avevamo definito l'incertezza semplicemente come $\log_2 c$, dove c è il numero di casi possibili. Questa definizione funziona correttamente quando tutti gli esiti sono equiprobabili. Ma se le probabilità sono diverse, $\log_2 c$ non cattura l'incertezza reale.

Consideriamo due partite di calcio:

Partita	Vittoria casa	Pareggio	Vittoria trasferta
Inter – Milan	30%	40%	30%
Real Madrid – Cagliari	90%	8%	2%

Entrambe le partite hanno tre possibili esiti, e quindi $\log_2 3 \simeq 1{,}585$ bit secondo la definizione basata sul solo conteggio. Eppure chiunque capisce che la prima è molto più incerta della seconda. La differenza sta nella distribuzione delle probabilità: nella prima le quote sono simili, nella seconda sono fortemente sbilanciate. Abbiamo dunque bisogno di una misura che restituisca un valore alto quando le probabilità sono simili tra loro e basso quando una domina sulle altre.

5.2. Definizione di entropia

La misura corretta dell'incertezza è la **media pesata** dell'informazione associata a ciascun esito, dove il peso di ogni esito è la sua probabilità. Questa grandezza prende il nome di **entropia**.

Definizione 5.1— Entropia di Shannon

Sia X una variabile casuale che può assumere gli esiti $x_1, x_2, \ldots, x_n$ con probabilità $p_1, p_2, \ldots, p_n$ (con $\sum_i p_i = 1$). L'*entropia* di X è definita come

$$H(X) = -\sum_{i=1}^{n} p_i \log_2 p_i = \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i}.$$

Si misura in bit ed è sempre non negativa.

La giustificazione è la seguente: prima di osservare l'esito di X, non sappiamo quale dei valori x_i si manifesterà. Possiamo però calcolare la quantità di informazione che *ci aspettiamo* di ottenere. Dato che il valore x_i si verifica con probabilità p_i e, quando si verifica, ci trasmette $-\log_2 p_i$ bit, l'informazione media attesa è precisamente $H(X)$.

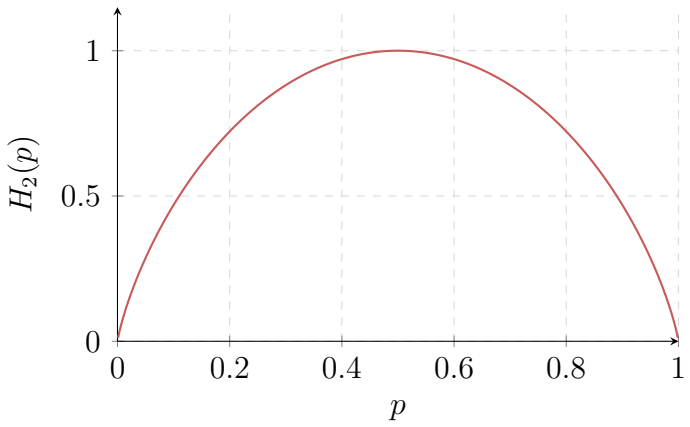
L'entropia è alta quando le probabilità sono simili tra loro (massima incertezza), ed è bassa quando una probabilità domina sulle altre (bassa incertezza). In particolare, se tutti gli n esiti hanno la stessa probabilità $1/n$, allora $H(X) = \log_2 n$, che coincide con la nostra definizione elementare di incertezza; se invece un esito ha probabilità 1 (tutti gli altri zero), $H(X) = 0$: sappiamo già cosa accadrà.

5.3. Andamento dell'entropia binaria

Il caso più semplice è quello in cui ci sono solo due esiti possibili, con probabilità p e $1 - p$. La **entropia binaria**

$$H_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

ha l'andamento riportato nel grafico seguente.



La curva ha alcune caratteristiche che è utile leggere esplicitamente. Il massimo si trova in $p = 0{,}5$ e vale $H_2(0{,}5) = 1$ bit: quando i due esiti sono equiprobabili, non c'è alcuna ragione di preferire l'uno all'altro, e l'incertezza è massima. La curva è simmetrica rispetto a $p = 0{,}5$: il che è naturale, perché scambiare i ruoli di 0 e 1 non cambia l'incertezza. Man mano che p si avvicina a 0 oppure a 1, l'entropia decresce verso 0: in entrambi i casi uno dei due esiti è quasi certo, e l'incertezza è quasi nulla.

Esempio 5.1— Entropia delle partite

Calcoliamo l'entropia per Inter–Milan e Real–Cagliari:

$$H(\text{Inter-Milan}) = \frac{30}{100}\log_2\frac{100}{30} + \frac{40}{100}\log_2\frac{100}{40} + \frac{30}{100}\log_2\frac{100}{30} \simeq 1{,}57 \text{ bit},$$
$$H(\text{Real-Cagliari}) = \frac{90}{100}\log_2\frac{100}{90} + \frac{8}{100}\log_2\frac{100}{8} + \frac{2}{100}\log_2\frac{100}{2} \simeq 0{,}54 \text{ bit}.$$

Il valore dell'entropia conferma quantitativamente l'intuizione: Inter–Milan è quasi tre volte più incerta di Real–Cagliari.

6. Ritorno al puzzle: strategia ottimale di pesata

Armati della definizione di entropia, possiamo ora tornare al problema delle dodici palline e affrontarlo in modo rigoroso.

6.1. Incertezza iniziale

Abbiamo 24 possibilità equiprobabili (12 palline $\times$ 2 modi di essere diversa). Poiché le probabilità sono uguali, l'entropia iniziale è semplicemente

$$H(X) = \log_2 24 \simeq 4{,}58 \text{ bit}.$$

Questo è il totale di informazione che dobbiamo acquisire per risolvere il puzzle. Qualunque strategia adottiamo, non potremo risolverlo con meno informazione di questa.

6.2. Informazione ottenuta da una pesata

Ogni pesata ha tre possibili esiti: il primo piatto è più pesante ($+$), il secondo è più pesante ($-$), oppure i due piatti sono in equilibrio ($=$). Indichiamo con N_+, N_- e $N_=$ il numero di possibilità ancora valide dopo ciascun esito, con il vincolo $N_+ + N_- + N_= = 24$.

Poiché le 24 possibilità iniziali sono equiprobabili, la probabilità di ciascun esito è proporzionale al numero di possibilità che esso lascia aperte. Di conseguenza, l'entropia residua *media* dopo la pesata è

$$H(X|e) = \frac{1}{24}\left(N_+\log_2 N_+ + N_-\log_2 N_- + N_=\log_2 N_=\right),$$

e l'informazione ottenuta dalla pesata è $I(X;e) = H(X) - H(X|e)$.

6.3. La pesata ottimale: bilanciare gli esiti

Vogliamo massimizzare l'informazione ottenuta da ciascuna pesata, ovvero minimizzare l'entropia residua $H(X|e)$. La funzione $f(t) = t\log_2 t$ è convessa — la sua curva piega verso l'alto, come una parabola. Per le funzioni convesse vale una legge intuitiva: la somma $f(N_+) + f(N_-) + f(N_=)$ è tanto più piccola quanto più i tre valori sono vicini tra loro. Il suo minimo, tenendo fissa la somma $N_+ + N_- + N_= = C$, si

raggiunge quando $N_+ = N_- = N_= = C/3$, ovvero quando le possibilità sono distribuite in modo perfettamente uniforme tra i tre esiti. Appiattire la distribuzione dei casi tra gli esiti riduce l'incertezza media quanto più possibile.

Proposizione 6.1— Principio della pesata ottimale

La pesata che massimizza l'informazione media acquisita è quella che suddivide le possibilità ancora valide nel modo più bilanciato possibile, assegnando lo stesso numero di possibilità a ciascuno dei tre esiti. In questo caso, l'entropia residua è $H(X|e) = \log_2(C/3)$, dove C è il numero totale di possibilità prima della pesata.

Per avere un'idea numerica, consideriamo il caso semplificato $C = 9$: nella tabella seguente riportiamo $H(X|e)$ al variare di N_+ e N_- (con $N_= = 9 - N_+ - N_-$):

$N_+ \backslash N_-$	1	2	3	4	5	6	7
1	2.183	1.946	1.818	1.778	1.818	1.946	2.183
2	1.946	1.734	1.639	1.639	1.734	1.946	–
3	1.818	1.639	**1.585**	1.639	1.818	–	–
4	1.778	1.639	1.639	1.778	–	–	–
5	1.818	1.734	1.818	–	–	–	–

Il valore minimo (in grassetto) si trova in $N_+ = N_- = N_= = 3$, confermando il principio enunciato. Il valore $1{,}585 = \log_2 3$ corrisponde all'entropia nel caso di tre possibilità equiprobabili.

6.4. Costruzione della strategia: prima pesata

Le possibilità iniziali sono 24, per cui la pesata ottimale deve lasciare $24/3 = 8$ possibilità per ciascun esito. L'incertezza residua dopo la prima pesata sarà $H(X|e_1) = \log_2 8 = 3$ bit.

Per ottenere esattamente 8 possibilità per ogni esito, è sufficiente porre 4 palline (a, b, c, d) sul primo piatto e 4 palline (e, f, g, h) sul secondo:

- se il primo piatto è più pesante $(+)$: le 8 possibilità sono $\{a+, b+, c+, d+, e-, f-, g-, h-\}$;
- se il secondo piatto è più pesante $(-)$: le 8 possibilità sono $\{a-, b-, c-, d-, e+, f+, g+, h+\}$;
- se i piatti sono in equilibrio $(=)$: le 8 possibilità rimanenti riguardano le quattro palline non pesate $\{i+, j+, k+, l+, i-, j-, k-, l-\}$.

6.5. Seconda e terza pesata

Dopo la prima pesata rimangono 8 possibilità. La suddivisione più bilanciata possibile tra tre esiti è $\{3, 3, 2\}$ (poiché 8 non è divisibile per 3), che porta a un'incertezza residua di

$$H(X|e_1, e_2) = \frac{1}{8}\left(3\log_2 3 + 3\log_2 3 + 2\log_2 2\right) \simeq 1{,}44 \text{ bit}.$$

Dopo la seconda pesata rimangono 2 o 3 possibilità. In entrambi i casi, una terza pesata opportunamente organizzata azzera completamente l'incertezza.

Osservazione 6.1— Il problema è risolubile in tre pesate

L'incertezza iniziale è $\log_2 24 \simeq 4{,}58$ bit. Una singola pesata può fornire al massimo $\log_2 3 \simeq 1{,}585$ bit di informazione. In due pesate si possono ottenere al massimo $2\log_2 3 \simeq 3{,}17$ bit, insufficienti a coprire i 4,58 bit. Con tre pesate, il massimo ottenibile è $3\log_2 3 \simeq 4{,}75$ bit, che supera l'incertezza iniziale. Tre pesate sono quindi il minimo necessario, e come abbiamo visto esistono strategie che le rendono sufficienti.

7. Entropia e linguaggio naturale

Le idee sviluppate nella teoria dell'informazione trovano applicazioni affascinanti nello studio del linguaggio naturale. In questa sezione esploriamo come l'entropia e la probabilità condizionata permettano di caratterizzare quantitativamente la struttura dell'italiano.

7.1. La ridondanza del linguaggio

Prima di affrontare i dettagli tecnici, consideriamo il seguente esperimento informale. Il testo qui sotto presenta numerose omissioni: provate a leggerlo.

> *"Il v_nt_, ve__ndo _n cit_à da l__tan_, le p__ta d_ni in___sueti, di cui s'a____gono so_o po__e an_me sen__bili, come i raf___ddati del fi_no, che sta__utisc_no per pol__ni d_ fi_ri d'al_re te_re."*
>
> (da *Marcovaldo* di Italo Calvino)

La frase è comprensibile, nonostante le numerosissime omissioni. Questo avviene perché la lingua italiana è fortemente **ridondante**: conoscendo il contesto, siamo in grado di prevedere con buona accuratezza i caratteri mancanti.

7.2. Entropia dei caratteri in italiano

Per quantificare questa ridondanza, costruiamo un semplice *modello statistico* dell'italiano. L'idea è di prendere un testo sufficientemente lungo, eliminare tutti i caratteri non alfabetici e gli spazi, e calcolare la frequenza relativa di ogni carattere.

Considerando la traduzione italiana de *Lo Hobbit* di J.R.R. Tolkien (546264 caratteri complessivi), la lettera "a" compare circa 50669 volte (probabilità $\approx 0{,}092$), la "b" compare 6477 volte (probabilità $\approx 0{,}012$), e così via. L'entropia calcolata su questi 27 simboli (26 lettere più lo spazio) risulta

$$H(X) \simeq 3{,}965 \text{ bit},$$

mentre l'entropia massimale nel caso equiprobabile sarebbe $H'(X) = \log_2 27 \simeq 4{,}755$ bit.

La **ridondanza del linguaggio** è

$$R = 1 - \frac{H(X)}{H'(X)} \simeq 0{,}166 :$$

circa il 16% della capacità informativa dell'alfabeto non viene sfruttata dalla distribuzione reale.

7.3. Entropia condizionata: l'effetto del contesto

Ma nella predizione del prossimo carattere, non utilizziamo soltanto la frequenza assoluta di ogni lettera: teniamo conto anche del contesto. Un correttore automatico, per esempio, dopo aver letto una "q" si aspetta quasi certamente una "u".

Formalmente, anziché le probabilità marginali $p(x)$, utilizziamo le **probabilità condizionate** $p(x|c)$, dove c è l'ultimo carattere letto. L'incertezza sul prossimo carattere è misurata dall'**entropia condizionata**

$$H(X|c) = -\sum_x p(x|c) \log_2 p(x|c).$$

Per esempio, se l'ultimo carattere letto è una "m", l'entropia condizionata risulta $H(X|\text{'m'}) \simeq 2{,}657$ bit, inferiore all'entropia marginale di 3,965 bit: la riduzione dell'incertezza è di circa il 33%. La media pesata su tutti i possibili caratteri precedenti dà $H(X|c) \simeq 3{,}148$ bit, il che corrisponde a una riduzione media del 21%.

7.4. Memoria più lunga, incertezza più bassa

Il ragionamento si estende al caso in cui si tengano conto di più caratteri precedenti. Dai dati de *Lo Hobbit* si ottiene:

Contesto considerato	Entropia residua
Nessuno (marginale)	$H(X) \simeq 3{,}965$ bit
Ultimo 1 carattere	$H(X\|c) \simeq 3{,}148$ bit
Ultimi 2 caratteri	$H(X\|c_1c_2) \simeq 2{,}651$ bit
Ultimi 3 caratteri	$H(X\|c_1c_2c_3) \simeq 2{,}136$ bit
Ultimi 4 caratteri	$H(X\|c_1c_2c_3c_4) \simeq 1{,}749$ bit

La tendenza è chiara: più lunga è la memoria del contesto, minore è l'incertezza sul prossimo carattere.

7.5. Generazione di testo artificiale

Le dipendenze statistiche della lingua possono essere usate "al contrario", per *generare* testo che assomiglia all'italiano. Il procedimento è semplice: si genera un carattere alla volta, ciascuno campionato secondo la distribuzione di probabilità condizionata al contesto precedente. Con sole probabilità marginali si ottiene una sequenza casuale di lettere; con contesti di 2 o 3 caratteri il testo inizia a contenere sillabe e parole riconoscibili; con contesti più lunghi la struttura diventa sorprendentemente plausibile.

7.6. Dall'analisi dei caratteri all'analisi delle parole

L'analisi statistica può essere ripetuta a livello di *parole*. Nel corpus de *Lo Hobbit* si contano **10837 parole distinte**. L'entropia massimale (distribuzione uniforme) è $H'(X) = \log_2 10837 \simeq 13{,}403$ bit; quella effettiva è $H(X) \simeq 10{,}047$ bit, con una ridondanza del 25%. Al crescere del contesto condizionante, la riduzione è impressionante:

Contesto (parole precedenti)	H (bit)	Riduzione
Nessuno (marginale)	10,047	–
Ultima 1 parola	4,951	50,7%
Ultime 2 parole	1,297	87,1%
Ultime 3 parole	0,207	97,9%
Ultime 4 parole	0,037	99,6%
Ultime 5 parole	0,008	99,9%

Conoscere le ultime due parole riduce l'incertezza sulla parola successiva di oltre l'87%; conoscere le ultime tre la porta quasi a zero. Questo riflette la struttura fortemente vincolata della lingua naturale.

Il valore a cui la curva dell'entropia condizionata tende al crescere della lunghezza del contesto prende il nome di **entropia del linguaggio**: esso rappresenta l'incertezza residua "intrinseca" della lingua, quella che non può essere eliminata nemmeno con una memoria infinita del contesto.

7.7. I moderni modelli del linguaggio

I **modelli del linguaggio di grandi dimensioni** (in inglese *Large Language Models*, LLM), come quelli alla base di ChatGPT, Gemini o Claude, sono la realizzazione attuale più avanzata di questa idea. Al loro cuore si trova esattamente il problema che abbiamo studiato: stimare la distribuzione di probabilità condizionata $p(w \mid w_1, w_2, \ldots, w_k)$ della prossima parola (o, più precisamente, del prossimo *token*) dato un contesto di k elementi precedenti.

La sfida è quella delle dimensioni. Un LLM moderno opera su un vocabolario di circa 50 000 token e su finestre di contesto di migliaia di token. Una tabella esaustiva delle probabilità condizionate — del tipo che abbiamo costruito per il corpus dello Hobbit — richiederebbe, anche solo per un contesto di lunghezza 20, un numero di voci dell'ordine di $50000^{20} = 5^{20} \times 10000^{20} \approx 10^{14} \times 10^{80} = 10^{94}$: più degli atomi nell'intero universo. Questo approccio è assolutamente fuori portata per qualsiasi memoria disponibile, oggi o in futuro.

La soluzione è sostituire la tabella con un **algoritmo**: una *rete neurale*, ovvero una funzione matematica con miliardi di parametri (i *pesi* della rete), che prende in ingresso la sequenza dei token precedenti e produce in uscita una distribuzione di probabilità sull'intero vocabolario. La rete non *memorizza* le distribuzioni: le *calcola*, sfruttando strutture matematiche (moltiplicazioni di matrici, funzioni non lineari, meccanismi di attenzione) che le consentono di approssimare in modo compatto funzioni condizionate complesse.

I pesi della rete vengono determinati durante la fase di **addestramento**: si mostra alla rete un corpus di centinaia di miliardi di token (testi scritti, codice sorgente, pagine web) e si modificano iterativamente i pesi in modo da massimizzare

la probabilità assegnata dalla rete alle parole effettivamente osservate. In termini formali, si minimizza la *cross-entropia* tra la distribuzione del modello e quella del testo reale — una grandezza strettamente legata all'entropia di Shannon: minimizzare la cross-entropia significa portare l'entropia del modello il più vicino possibile all'entropia reale della lingua.

Il risultato sorprendente — e non ancora del tutto compreso — è che da questo processo di stima statistica emergano capacità che vanno ben oltre la pura previsione del prossimo token: comprensione del contesto, ragionamento per analogia, capacità di traduzione, sintesi e conversazione. Il filo che connette le tavole di frequenza dello Hobbit e i miliardi di parametri di un LLM moderno passa interamente per la teoria dell'informazione.

8. Un secondo puzzle: indovinare un numero

Il problema delle palline ha mostrato come l'entropia governi il numero minimo di pesate necessarie, ciascuna con tre possibili esiti. Vale la pena esaminare un puzzle analogo in cui le domande hanno soltanto due possibili risposte: questo ci permetterà di mettere in luce un legame diretto con uno dei più importanti algoritmi dell'informatica.

8.1. Formulazione del problema

Un giocatore sceglie segretamente un numero intero x compreso tra 1 e $N = 10^6$. Il nostro obiettivo è individuare x ponendo domande della forma "$x < k$?", alle quali il giocatore risponde soltanto *Sì* oppure *No*. Ciascuna domanda ha dunque esattamente due possibili esiti.

8.2. L'incertezza iniziale e il limite inferiore

Poiché le 10^6 possibilità sono equiprobabili, l'entropia iniziale è

$$H(X) = \log_2 10^6 \simeq 19{,}931 \text{ bit}.$$

Ogni domanda ha al più due esiti, e quindi può fornire al massimo $\log_2 2 = 1$ bit di informazione. Ne deriva un limite inferiore immediato:

Proposizione 8.1— Limite inferiore per l'indovina-numero

Qualunque strategia si adotti, nessun algoritmo può individuare x in meno di $\lceil \log_2 N \rceil$ domande. Per $N = 10^6$, questo vale $\lceil 19{,}931 \rceil = 20$ domande.

8.3. La domanda ottimale: il valore mediano

La domanda "$x < k$?" suddivide le possibilità correnti in due sottoinsiemi: se la risposta è *Sì*, rimangono $N_S = k-1$ possibilità; se è *No*, rimangono $N_N = N-k+1$. Come per le palline, l'informazione media è massima quando i due sottoinsiemi sono il più possibile uguali. Una domanda sbilanciata come "$x < 2$?" fornisce praticamente zero informazione ($\simeq 2 \times 10^{-5}$ bit), mentre la domanda sul valore mediano "$x < 500001$?" fornisce esattamente 1 bit, il massimo possibile.

8.4. La strategia ottimale: la ricerca binaria

Applicando la scelta del valore mediano ad ogni passo si ottiene il celebre algoritmo di **ricerca binaria**: ad ogni domanda lo spazio delle possibilità viene dimezzato, fornendo esattamente 1 bit di informazione. Dopo t domande, il numero di possibilità residue è $\simeq N/2^t$ e per azzerare l'incertezza servono $t = \lceil \log_2 N \rceil = 20$ domande, che raggiunge esattamente il limite inferiore derivato dall'entropia.

9. Generalizzazione: l'ordinamento per confronti

Lo stesso schema di ragionamento si applica al problema di *ordinare* un insieme di valori, uno dei più fondamentali in informatica.

9.1. Ordinare come problema di informazione

Supponiamo di avere n oggetti distinti in ordine sconosciuto e di volerli disporre in ordine crescente effettuando confronti binari del tipo "$a < b$?". Le configurazioni iniziali possibili sono $n!$; l'incertezza iniziale è $H(X) = \log_2(n!)$. Ogni confronto ha due esiti e può quindi fornire al massimo 1 bit di informazione.

9.2. Il limite imposto dall'entropia e l'approssimazione di Stirling

Ne deriva un limite inferiore universale: qualunque algoritmo di ordinamento basato su confronti necessita di almeno $\lceil \log_2(n!) \rceil$ confronti nel caso peggiore.

Il numero $n!$ cresce molto rapidamente con n (per $n = 20$ è dell'ordine di 10^{18}), ma il suo logaritmo è descrivibile in modo semplice. L'*approssimazione di Stirling* afferma che, per grandi valori di n, il fattoriale si comporta come $n! \approx (n/e)^n \sqrt{2\pi n}$, dove $e \simeq 2{,}718$ è la base del logaritmo naturale. Prendendo il $\log_2$ e ricordando che $\log_2 e \simeq 1{,}443$, si ottiene

$$\log_2(n!) \approx n \log_2 n - n \log_2 e + \tfrac{1}{2} \log_2(2\pi n) \approx n \log_2 n - 1{,}443\, n,$$

poiché il termine $\frac{1}{2} \log_2(2\pi n)$ cresce molto più lentamente di n. Questa approssimazione è già ottima per $n \geq 20$ e ci dice che il limite inferiore per l'ordinamento è dell'ordine di $n \log_2 n$.

9.3. Mergesort e la vicinanza al limite

L'algoritmo di **mergesort** effettua un numero di confronti pari a $n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$, che si avvicina al limite informativo tanto più quanto più n è grande:

n	10	100	1000	5000	10000	50000	100000
Confronti necessari	21	522	8525	54227	118452	708349	1516696
Confronti (mergesort)	25	573	8977	56809	123617	734465	1568929
Gap %	16	8,9	5,0	4,5	4,2	3,6	3,3

Al crescere di n, il mergesort si avvicina sempre più al limite fissato dall'entropia: il gap percentuale scende da circa il 16% per $n = 10$ a poco più del 3% per $n = 100000$. È un risultato notevole: un algoritmo concepito senza nessun riferimento esplicito alla teoria dell'informazione si dimostra asintoticamente quasi ottimale rispetto al limite teorico imposto dall'entropia.

Osservazione 9.1— Il principio generale

L'entropia fornisce un **limite inferiore universale** al numero di operazioni necessarie per risolvere qualunque problema di ricerca o decisione basato su domande a esiti limitati. Che si tratti di palline su una bilancia, di un numero da indovinare o di un insieme da ordinare, la strategia ottimale è sempre quella che massimizza il guadagno informativo a ogni passo, bilanciando il più possibile il numero di casi compatibili con ciascun esito.

10. Verso una teoria della codifica: sorgente, canale e destinatario

Fino a questo punto ci siamo concentrati sulla *misura* dell'informazione: quanti bit porta un evento, quanta è l'incertezza prima di un'osservazione, come questa si riduce man mano che acquisiamo dati. Abbiamo visto che questi concetti governano i limiti fondamentali di problemi molto diversi — pesate, ricerca, ordinamento, linguaggio naturale. È arrivato il momento di chiedersi come l'informazione venga concretamente *rappresentata* e *trasmessa*.

Il problema è così antico quanto la comunicazione stessa. Come può Alice, osservato un certo evento, comunicarlo a Bruno in modo efficiente e affidabile? La risposta, nella sua forma moderna e matematicamente rigorosa, nasce da un singolo articolo: *A Mathematical Theory of Communication*, pubblicato da Claude Shannon sui *Bell System Technical Journal* nel 1948. Shannon era un matematico e ingegnere che lavorava ai Bell Telephone Laboratories — il celebre laboratorio di ricerca della AT&T dove erano stati inventati il transistor e decine di altre tecnologie fondamentali. In poche decine di pagine, Shannon dimostrò qualcosa che molti ritenevano impossibile: esistono limiti esatti e raggiungibili tanto alla compressione dell'informazione quanto alla sua trasmissione su canali rumorosi. Questi limiti sono fissati dall'entropia, e la loro esattezza è ciò che rende la teoria di Shannon così potente. Da quella pubblicazione ha preso il via l'intera disciplina della teoria dell'informazione e della codifica, alla base di ogni sistema di comunicazione digitale moderno.

10.1. Lo schema astratto della comunicazione

La trasmissione dell'informazione da Alice a Bruno avviene in due fasi distinte. Prima i due si accordano su come rappresentare i possibili esiti: a ogni esito deve corrispondere un messaggio ben definito, e questo insieme di regole condivise prende il nome di **codice**. Poi avviene la comunicazione vera e propria: Alice, osservato un certo esito, lo trasforma nel messaggio corrispondente, lo invia attraverso un **canale**, e Bruno, ricevuto il messaggio, ne ricostruisce l'esito originale.

Questo processo si articola in tre stadi ben distinti, illustrati in Figura **??**:

1. **Sorgente.** L'entità che genera l'informazione: un testo, un segnale audio, un'immagine, l'esito di un esperimento, la sequenza di istruzioni di un programma.

2. **Codificatore e canale.** Il codificatore trasforma il messaggio prodotto dalla sorgente in una rappresentazione adatta al canale fisico disponibile; il canale lo trasmette o lo memorizza, eventualmente introducendo rumore o errori.

3. **Decodificatore e destinatario.** Il decodificatore inverte la trasformazione del codificatore e consegna il messaggio al destinatario, idealmente senza alcuna perdita di informazione.

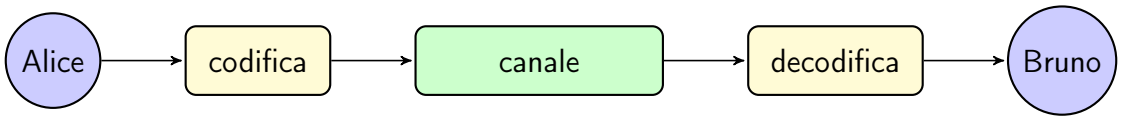


Figura 1: Schema della comunicazione da Alice a Bruno.

Questo schema astratto è universale: si applica identicamente a un testo digitale trasmesso su Internet, a un segnale audio compresso in formato MP3, a un'immagine memorizzata su disco, a una sequenza genomica conservata in una banca dati biologica, a un messaggio radio inviato da una sonda spaziale. In tutti questi casi compaiono le stesse tre componenti — sorgente, canale, destinatario — e gli stessi due problemi fondamentali: come rappresentare l'informazione in modo efficiente, e come proteggerla dagli errori introdotti dal canale.

10.2. Le due grandi sfide della codifica

Dal punto di vista di Bruno, prima di ricevere il messaggio esiste un'incertezza sull'esito osservato da Alice: questa incertezza è misurabile in bit. Quando il messaggio viene ricevuto e decodificato correttamente, l'incertezza si azzera e l'informazione viene acquisita. Nel processo compaiono quindi due quantità distinte, entrambe misurate in bit: l'incertezza iniziale (l'informazione potenzialmente acquisibile) e la lunghezza del messaggio effettivamente trasmesso. Il rapporto tra queste due quantità è al centro di tutta la teoria della codifica.

Le operazioni svolte dal codificatore possono essere ricondotte a due categorie concettuali:

- La **codifica di sorgente** riduce la dimensione del messaggio eliminando ridondanze presenti nell'informazione originaria: è il problema della *compressione*, affrontato da formati come ZIP, MP3 o JPEG.
- La **codifica di canale** introduce deliberatamente ridondanza controllata per consentire il rilevamento e la correzione degli errori introdotti dal canale: è il problema della *protezione dall'errore*, alla base dei codici usati nelle trasmissioni satellitari, nei dischi rigidi o nelle reti mobili.

Osservazione 10.1— Due obiettivi opposti, stessa teoria

Compressione e protezione dagli errori sembrano in apparente contraddizione: la prima elimina ridondanza, la seconda la introduce. In realtà sono governate dagli stessi limiti informativi. Comprimere significa eliminare ridondanza *non strutturata*; proteggere significa introdurre ridondanza *strutturata e controllata*. In un sistema di comunicazione reale le due operazioni si applicano in sequenza: prima si comprime il messaggio, poi si aggiunge ridondanza per il canale.

Nelle sezioni che seguono svilupperemo entrambi questi aspetti, partendo dalla codifica di sorgente e dai suoi limiti teorici, per poi affrontare la codifica di canale e i codici correttori di errori.

11. Codifica di sorgente

Con il modello sorgente-canale-destinatario ben in mente, possiamo ora affrontare il primo dei due problemi fondamentali: come rappresentare l'informazione prodotta dalla sorgente con il minor numero medio di bit possibile. Questo è il problema della *codifica di sorgente*, e il risultato centrale — il teorema di Shannon — ne fissa il limite con precisione assoluta.

11.1. Il modello della sorgente

Formalizziamo la situazione. Alice è la sorgente: a ogni istante emette un simbolo scelto da un alfabeto finito $S = \{s_1, s_2, \ldots, s_m\}$, dove il simbolo s_i viene emesso con probabilità p_i. La variabile casuale X che descrive il prossimo simbolo emesso ha entropia $H(X) = -\sum_i p_i \log_2 p_i$. L'obiettivo è rispondere alla domanda: qual è il numero *minimo medio* di bit necessari per trasmettere i simboli di questa sorgente a Bruno?

Una **codifica** associa a ogni simbolo s_i una parola binaria $c(s_i)$ di lunghezza l_i bit. Il flusso prodotto dalla sorgente viene trasformato in una sequenza di bit ottenuta concatenando le parole di codice dei simboli emessi.

11.2. Codici e decodificabilità univoca

La concatenazione delle parole di codice avviene *senza separatori*: il canale digitale non ha spazi tra una parola e l'altra. Questo è essenziale per l'efficienza, ma pone un problema: come fa Bruno a sapere dove finisce una parola di codice e dove inizia la successiva?

Con un codice a lunghezza fissa la risposta è ovvia — ogni l bit formano un simbolo. Con lunghezze variabili, non è più così. Si potrebbe inserire un separatore tra le parole, ma questo spreca bit preziosi. La soluzione elegante è imporre un vincolo strutturale: un codice si dice **a prefisso** (o *istantaneo*) se nessuna parola di codice è uguale all'inizio di un'altra. Con questa proprietà, Bruno può decodificare bit per bit: non appena i bit letti formano una parola valida, il simbolo è completo. Non c'è mai bisogno di guardare "avanti", e non c'è mai ambiguità.

Esempio 11.1— Codice prefisso e non-prefisso

Il codice $\{a \mapsto \texttt{0},\ b \mapsto \texttt{10},\ c \mapsto \texttt{11}\}$ è a prefisso: nessuna parola è inizio di un'altra. Ricevendo `01011`, Bruno legge `0`$\to a$, poi `10`$\to b$, poi `11`$\to c$: decodifica univoca e immediata.
Al contrario, il codice $\{a \mapsto \texttt{0},\ b \mapsto \texttt{01},\ c \mapsto \texttt{011}\}$ non è a prefisso: `0` è prefisso di `01` e `011`. Ricevendo `011`, Bruno non sa se leggere a-b oppure a-c oppure aspettare altri bit.

11.3. La lunghezza media e il teorema di Shannon

La **lunghezza media** di un codice misura il costo, in bit per simbolo, della codifica:

$$L(c, X) = \sum_{i=1}^{m} p_i \, l_i.$$

Un codice è tanto migliore quanto più bassa è la sua lunghezza media: minori bit per simbolo significano messaggi più corti, trasmissioni più veloci e meno spazio di memorizzazione.

Il risultato fondamentale della teoria — noto come *primo teorema di Shannon* — stabilisce il limite invalicabile:

Proposizione 11.1— Teorema di Shannon sulla codifica di sorgente

Per ogni codice univocamente decodificabile vale

$$L(c, X) \geq H(X).$$

Inoltre, per ogni $\varepsilon > 0$ esistono codici con $L(c, X) \leq H(X) + \varepsilon$, codificando opportunamente blocchi di simboli successivi; nel limite,

$$\lim_{k \to \infty} L(c, X^k) = H(X).$$

In altre parole, l'entropia $H(X)$ è il *numero minimo medio di bit per simbolo* necessario per rappresentare senza perdita l'uscita della sorgente. Non si può fare meglio: qualunque tentativo di comprimere al di sotto di $H(X)$ bit per simbolo porta inevitabilmente a perdere informazione, perché due messaggi distinti finirebbero per avere la stessa descrizione, rendendo impossibile la decodifica.

Il motivo per cui il teorema richiede la codifica a blocchi è legato all'integrità dei bit: le lunghezze l_i devono essere interi, e l'arrotondamento introduce un overhead di al più 1 bit per simbolo. Codificando blocchi di k simboli, questo overhead si distribuisce su k simboli e diventa trascurabile per simbolo al crescere di k.

11.4. Compressione come trasformazione

Una codifica senza perdita può essere vista come una trasformazione $X \to Y$, dove Y è la rappresentazione binaria di X. Poiché conoscendo Y possiamo ricostruire X con certezza, si ha $H(X \mid Y) = 0$. Usando la decomposizione dell'entropia $H(X) = I(X;Y) + H(X|Y)$, si ottiene $I(X;Y) = H(X)$: la rappresentazione binaria porta esattamente tanta informazione quanta ne porta il simbolo originale.

Comprimere senza perdita significa dunque non distruggere informazione, ma soltanto eliminarne la *ridondanza*: si cambia la rappresentazione mantenendo il contenuto informativo intatto.

12. Codici a lunghezza fissa

Il codice più semplice possibile è quello a **lunghezza fissa**: ogni simbolo dell'alfabeto riceve una parola di codice della stessa lunghezza l. Questo approccio ha il vantaggio della semplicità assoluta, ma come vedremo può essere molto inefficiente per sorgenti con distribuzione non uniforme.

12.1. Costruzione e funzionamento

Con K simboli nell'alfabeto, per distinguere tra K casi è necessario un numero di bit pari a $l = \lceil \log_2 K \rceil$: con l bit si formano $2^l \geq K$ parole distinte. Poiché tutte le parole hanno la stessa lunghezza, la decodifica è immediata: Bruno suddivide il flusso di bit in blocchi di l bit, ciascuno corrispondente a un simbolo, senza ambiguità.

Esempio 12.1— Codifica a lunghezza fissa con $K = 4$

Con quattro simboli $\{a, b, c, d\}$ si ha $l = \lceil \log_2 4 \rceil = 2$ bit:

s_i	a	b	c	d
$c(s_i)$	00	01	10	11

Alice trasmette $c\,a\,d\,b$, Bruno riceve `10001101` e decodifica correttamente leggendo blocchi di 2 bit. Per $K = 5$ sarebbero necessari $l = 3$ bit, con $2^3 = 8$ parole disponibili ma solo 5 utilizzate: le 3 rimanenti non corrispondono ad alcun simbolo.

12.2. Efficienza e limiti

L'**efficienza** del codice a lunghezza fissa si misura come

$$\eta = \frac{H(X)}{l} = \frac{H(X)}{\lceil \log_2 K \rceil}.$$

Quando $\eta = 1$ il codice usa esattamente tanti bit quanta è l'incertezza della sorgente; quando $\eta < 1$ vi è uno spreco. Il codice è ottimale soltanto quando i simboli sono equiprobabili *e* K è una potenza di 2. In ogni altro caso vi è inefficienza, che può diventare drammatica per distribuzioni molto asimmetriche.

Esempio 12.2— Il caso estremo

Consideriamo un alfabeto $\{a, b, c, d\}$ con $p(a) = 0{,}99$ e $p(b) = p(c) = p(d) = 0{,}003\overline{3}$. L'entropia è circa $H(X) \simeq 0{,}10$ bit per simbolo, ma il codice a lunghezza fissa richiede comunque $l = 2$ bit. L'efficienza è $\eta = 0{,}10/2 = 0{,}05$: si usano 20 volte più bit del necessario, con il 95% della capacità di trasmissione sprecato.

Questo esempio estremo non è un caso accademico: la distribuzione delle parole in qualsiasi testo naturale, o la distribuzione dei caratteri in quasi qualsiasi linguaggio, è fortemente asimmetrica. Per distribuzioni realistiche è quindi necessario ricorrere a codici a *lunghezza variabile*, che assegnino parole più corte ai simboli frequenti e parole più lunghe a quelli rari.

13. La codifica di Shannon-Fano

Il primo approccio sistematico ai codici a lunghezza variabile è la **codifica di Shannon-Fano**, proposta negli anni '40 da Shannon e in modo indipendente da Fano. Essa traduce il principio intuitivo "più frequente $\Rightarrow$ parola più corta" in un algoritmo ricorsivo che costruisce il codice dall'alto verso il basso.

13.1. L'idea: dividere bilanciando

Supponiamo di voler scegliere il primo bit delle parole di codice. Quel bit separa i simboli in due gruppi: quelli a cui verrà assegnato `0` e quelli a cui verrà assegnato `1`. Se i due gruppi hanno probabilità totale simile, il bit trasporta quasi 1 bit di informazione, usato al massimo della sua efficienza. Conviene dunque dividere l'alfabeto in due sottogruppi con probabilità totali il più possibile equilibrate, assegnare `0` al primo e `1` al secondo, e ripetere il procedimento ricorsivamente all'interno di ciascun sottogruppo.

13.2. L'algoritmo

L'algoritmo di Shannon-Fano procede come segue:

1. Ordinare i simboli per probabilità decrescente.
2. Dividere il gruppo corrente in due sottogruppi con probabilità totali il più possibile bilanciate.
3. Assegnare `0` al primo sottogruppo e `1` al secondo.
4. Ripetere il procedimento all'interno di ciascun sottogruppo, finché ogni gruppo contiene un solo simbolo.

La parola di codice di ogni simbolo è la concatenazione dei bit assegnati lungo il percorso di divisioni che lo porta alla sua foglia.

13.3. Il caso ideale: probabilità che sono potenze di $1/2$

Esempio 13.1— Alfabeto $\{a, b, c, d\}$ con probabilità $1/2, 1/4, 1/8, 1/8$

1ª divisione: $\{a\}$ (prob. $1/2$) vs. $\{b, c, d\}$ (prob. $1/2$) $\Rightarrow a \mapsto$ `0`.
2ª divisione (in $\{b, c, d\}$): $\{b\}$ (prob. $1/4$) vs. $\{c, d\}$ (prob. $1/4$) $\Rightarrow b \mapsto$ `10`.
3ª divisione (in $\{c, d\}$): $\{c\}$ vs. $\{d\}$, entrambi prob. $1/8 \Rightarrow c \mapsto$ `110`, $d \mapsto$ `111`.

La lunghezza media è $L = 1/2 \cdot 1 + 1/4 \cdot 2 + 1/8 \cdot 3 + 1/8 \cdot 3 = 1{,}75$ bit. Poiché $H(X) = 1{,}75$ bit, il codice è *esattamente* ottimale: ogni divisione è perfettamente bilanciata perché le probabilità sono potenze di $1/2$.

13.4. Un caso subottimale

Quando le probabilità non sono potenze di $1/2$, l'algoritmo può non trovare la soluzione ottimale, perché compie scelte localmente corrette che si rivelano globalmente non convenienti.

Esempio 13.2— Sorgente a cinque simboli

Con $p(a) = 0{,}35$, $p(b) = 0{,}25$, $p(c) = 0{,}20$, $p(d) = 0{,}15$, $p(e) = 0{,}05$, l'entropia è $H(X) \simeq 2{,}11$ bit.
La divisione più bilanciata al primo livello è $\{a, b\}$ (prob. 0,60) vs. $\{c, d, e\}$ (prob. 0,40). Procedendo ricorsivamente si ottiene $a \mapsto$ `00`, $b \mapsto$ `01`, $c \mapsto$ `10`,

$d \mapsto$ `110`, $e \mapsto$ `111`, con $L_{SF} = 2{,}20$ bit $> H(X) = 2{,}11$ bit.
Il problema è che a, con probabilità 0,35, ha contenuto informativo $-\log_2 0{,}35 \simeq 1{,}51$ bit: una parola da 1 o 2 bit basterebbe. Ma la prima divisione *obbliga* a a condividere il prefisso `0` con b, e così a riceve una parola da 2 bit anziché da 1. Shannon-Fano non "vede" oltre il livello corrente.

13.5. Il limite strutturale di Shannon-Fano

Shannon-Fano è un algoritmo *greedy*: ottimizza localmente, livello per livello, senza garanzia di ottimalità globale. Formalmente, produce sempre un codice prefisso con $H(X) \leq L_{SF} < H(X) + 1$, ma non garantisce che nessun altro codice prefisso possa fare meglio. Esiste un algoritmo che corregge questo difetto?

14. La codifica di Huffman

La risposta è sì, e ha una storia straordinaria. Nel 1951, David Huffman era uno studente di dottorato al MIT. Robert Fano — uno degli inventori del codice Shannon-Fano e tra i fondatori della teoria dell'informazione — stava tenendo un corso sull'argomento e propose ai suoi studenti una scelta: superare l'esame con una prova tradizionale, oppure dimostrare l'ottimalità di un codice prefisso come progetto finale. Huffman aveva quasi rinunciato e stava per prepararsi all'esame classico, quando gli venne in mente l'idea chiave: invece di partire dall'alto e dividere l'alfabeto, si poteva partire dal basso e *accorpare* i simboli meno probabili. Il codice che ne risultò era più efficiente di qualsiasi algoritmo noto, incluso quello di Shannon-Fano proposto dallo stesso Fano. Huffman aveva battuto il suo professore, e il codice porta il suo nome ancora oggi.

14.1. L'approccio dal basso verso l'alto

L'intuizione di Huffman è la seguente: in qualsiasi codice prefisso ottimale, i due simboli con probabilità più bassa avranno le parole di codice più lunghe. Anzi, si può dimostrare che devono essere "fratelli" nell'albero: le loro parole differiscono soltanto nell'ultimo bit. Se questo è necessariamente vero per il codice ottimale, ha senso collocarli subito in fondo all'albero, accorpandoli in un unico *simbolo virtuale* la cui probabilità è la somma delle due, e ripetere il procedimento. Invece di costruire l'albero dall'alto spezzando, si costruisce dal basso fondendo.

14.2. L'algoritmo di Huffman

L'algoritmo si descrive in cinque passi:

1. Partire da una lista di K nodi-foglia, uno per simbolo, etichettati con la rispettiva probabilità.
2. Scegliere i due nodi con probabilità più bassa.
3. Creare un nodo padre con probabilità pari alla loro somma; assegnare `0` al ramo di sinistra e `1` al ramo di destra.
4. Reinserire il nodo padre nella lista al posto dei due figli.

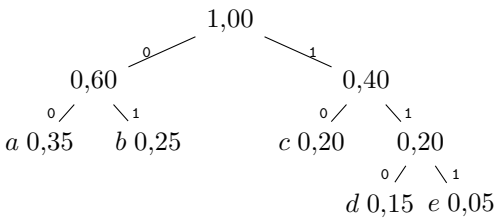
5. Ripetere finché la lista contiene un solo nodo: esso è la radice dell'albero.

La parola di codice di ogni simbolo è il cammino dalla radice alla sua foglia. Per un alfabeto di K simboli, l'algoritmo compie esattamente $K - 1$ fusioni.

14.3. Esempio: la sorgente a cinque simboli

Riprendiamo la sorgente $p(a) = 0{,}35$, $p(b) = 0{,}25$, $p(c) = 0{,}20$, $p(d) = 0{,}15$, $p(e) = 0{,}05$, su cui Shannon-Fano aveva prodotto $L_{SF} = 2{,}20$ bit.

Le fusioni procedono nell'ordine seguente: prima si fondono $e(0{,}05)$ e $d(0{,}15)$ nel nodo $de(0{,}20)$; poi $c(0{,}20)$ e $de(0{,}20)$ formano $cde(0{,}40)$; poi $b(0{,}25)$ e $a(0{,}35)$ formano $ab(0{,}60)$; infine $ab(0{,}60)$ e $cde(0{,}40)$ si fondono nella radice. L'albero risultante è:



Il codice risultante — $a \mapsto \texttt{00}$, $b \mapsto \texttt{01}$, $c \mapsto \texttt{10}$, $d \mapsto \texttt{110}$, $e \mapsto \texttt{111}$ — coincide in questo caso con quello di Shannon-Fano, con $L = 2{,}20$ bit. La coincidenza è dovuta alla struttura particolare di questa distribuzione.

14.4. Huffman batte Shannon-Fano

La superiorità di Huffman emerge quando le probabilità sono meno regolari. Consideriamo $p(A) = 0{,}35$, $p(B) = 0{,}17$, $p(C) = 0{,}17$, $p(D) = 0{,}16$, $p(E) = 0{,}15$, con entropia $H(X) \simeq 2{,}23$ bit.

Shannon-Fano sceglie come prima divisione $\{A, B\}$ vs. $\{C, D, E\}$ (la più bilanciata), per cui A e B finiscono entrambi con parole da 2 bit, e la lunghezza media risulta $L_{SF} = 2{,}31$ bit.

Huffman procede dal basso: accorpa E con D, poi B con C, poi $[DE]$ con $[BC]$, e infine A con $[BCDE]$. Il simbolo A risulta figlio diretto della radice, con una parola da un solo bit, per cui

$$L_{Huff} = 0{,}35 \cdot 1 + 0{,}65 \cdot 3 = 2{,}30 \text{ bit} < L_{SF} = 2{,}31 \text{ bit}.$$

Shannon-Fano aveva cercato la divisione bilanciata localmente; Huffman, costruendo dal basso, ha trovato la struttura globale ottimale, riconoscendo che A meritava una parola da 1 bit.

14.5. Ottimalità di Huffman e applicazioni pratiche

Proposizione 14.1— Teorema di Huffman (1952)

Tra tutti i codici prefisso su un dato alfabeto, il codice prodotto dall'algoritmo di Huffman ha la minima lunghezza media.

Il nucleo dell'argomento è il seguente: in qualunque codice prefisso ottimale, i due simboli con probabilità più bassa sono necessariamente fratelli. Di conseguenza,

si possono accorpare in un simbolo virtuale, riducendo il problema da K a $K-1$ simboli. Per induzione, la procedura greedy di Huffman è globalmente ottimale.

Huffman soddisfa il bound $H(X) \leq L_{Huff} < H(X)+1$, con la garanzia aggiuntiva che nessun altro codice prefisso può fare meglio. Per avvicinarsi ulteriormente a $H(X)$, la tecnica della *codifica a blocchi* raggruppa n simboli consecutivi come un'unica entità, portando la lunghezza media sotto $H(X)+1/n$.

La codifica di Huffman è presente in quasi tutti i formati di compressione moderni, spesso come fase finale di una catena più complessa. Nel formato **JPEG**, un'immagine viene prima suddivisa in blocchi di 8×8 pixel, trasformata con la *trasformata discreta del coseno* (che concentra l'informazione visiva nei coefficienti a bassa frequenza) e quantizzata; i coefficienti risultanti, con distribuzione fortemente asimmetrica, vengono poi compressi con Huffman. Nel formato **MP3**, il segnale audio viene analizzato in bande di frequenza tenendo conto di ciò che l'orecchio umano percepisce (alcune frequenze vengono semplificate senza che l'ascoltatore se ne accorga), e i valori risultanti vengono compressi con Huffman. Nel formato **ZIP** (e nel suo successore `gzip`), l'algoritmo *DEFLATE* prima individua sequenze ripetute nel testo sostituendole con riferimenti brevi (algoritmo di Lempel-Ziv), poi comprime i simboli rimanenti con Huffman. In tutti questi casi, Huffman garantisce che la fase finale di codifica si avvicini quanto possibile al limite dell'entropia locale dei simboli.

15. Oltre il canale perfetto: codifica di canale

Tutta la discussione sulla compressione ha assunto un'ipotesi comoda e spesso irrealistica: il canale di trasmissione è *perfetto*, ovvero Bruno riceve esattamente ciò che Alice ha inviato. In molti scenari reali, invece, i simboli trasmessi possono arrivare modificati: interferenze radio, supporti fisici difettosi, rumore elettronico, raggi cosmici che alterano singoli bit in memoria — tutti questi fenomeni possono corrompere il messaggio durante la trasmissione. Si apre così la seconda delle due grandi sfide della codifica: come garantire l'affidabilità della comunicazione su un canale rumoroso?

15.1. Il canale binario simmetrico

Il modello più semplice di canale rumoroso è il *canale binario simmetrico* (CBS), descritto da un unico parametro: la probabilità f che un singolo bit venga invertito durante la trasmissione. Con probabilità $1-f$ il bit arriva corretto; con probabilità f viene capovolto. Il modello è "simmetrico" perché la probabilità di errore è la stessa per $0 \to 1$ e per $1 \to 0$. Un f piccolo indica un canale poco rumoroso; un f vicino a 0,5 indica un canale quasi inutile, la cui uscita è praticamente indipendente dall'ingresso.

15.2. La ridondanza come risorsa

Come difendersi dal rumore? L'idea chiave è introdurre **ridondanza** nel messaggio trasmesso. Questo sembra in apparente contraddizione con la codifica di sorgente, che punta a eliminarla. In realtà non lo è: la ridondanza che la codifica di canale *introduce* è strutturata e controllata, e serve a creare vincoli aggiuntivi che consentono di individuare e correggere gli errori. Si può pensare ai bit ridondanti come a

"testimoni" che sorvegliano gruppi di bit informativi e segnalano quando qualcosa è andato storto.

15.3. Il codice a ripetizione

Il metodo più semplice è il **codice a ripetizione**: invece di trasmettere ogni bit una sola volta, lo si trasmette tre volte. Bruno riceve tre copie e sceglie il valore della maggioranza.

Esempio 15.1— Ripetizione tripla con $f = 0{,}1$

Con un canale che sbaglia ogni bit con probabilità $f = 0{,}1$, la probabilità che un terzetto venga decodificato erroneamente (almeno 2 errori su 3) è

$$P(\text{errore}) = \binom{3}{2} f^2(1-f) + f^3 = 3 \cdot 0{,}01 \cdot 0{,}9 + 0{,}001 \approx 0{,}028.$$

Si scende dal 10% al 2,8%: un miglioramento reale. Il costo è che per ogni bit informativo se ne trasmettono tre.

L'approccio è però brutalmente inefficiente. Per raggiungere probabilità di errore dell'ordine di 10^{-15} — necessaria, ad esempio, per la copia affidabile di un terabyte — sarebbero necessarie circa 68 ripetizioni, con un overhead del 6700%: del tutto impraticabile. Servono codici più intelligenti, che proteggano *blocchi* di bit aggiungendo ridondanza in modo strutturato.

16. Distanza di Hamming e codici a blocchi

Per costruire codici correttori di errori intelligenti è necessario dotarsi di uno strumento geometrico che misuri quanto due messaggi binari si discostano l'uno dall'altro.

16.1. La distanza di Hamming

Definizione 16.1— Distanza di Hamming

Dati due messaggi binari x e y della stessa lunghezza, la loro *distanza di Hamming* $\delta(x, y)$ è il numero di posizioni in cui differiscono.

Per esempio, $\delta(\texttt{0110}, \texttt{0010}) = 1$ e $\delta(\texttt{1100}, \texttt{0000}) = 2$. La distanza di Hamming misura precisamente il "numero di errori" che occorrono: un singolo errore sposta il messaggio di distanza 1, due errori indipendenti di distanza 2, e così via.

16.2. Codici a blocchi (n, k)

Un **codice a blocchi (n, k)** prende un messaggio di k bit e lo trasforma in una *parola di codice* di $n > k$ bit, aggiungendo $n - k$ bit di ridondanza. L'idea è che non tutte le 2^n sequenze di n bit siano parole di codice valide: solo 2^k di esse lo sono. Le sequenze rimanenti fungono da "spazio vuoto" tra le parole valide: se una sequenza ricevuta non è una parola valida, significa che è avvenuto un errore.

Il **tasso di codice** $R = k/n$ misura la fraction di bit informativi. Aggiungere ridondanza significa ridurre R; la chiave è scegliere la ridondanza in modo che sia il più possibile strutturata ed efficiente.

16.3. Rilevare e correggere errori

La capacità di un codice dipende dalla sua **distanza minima** $d_{\min}$, ovvero la minima distanza di Hamming tra due parole di codice valide distinte.

Proposizione 16.1— Regola generale per rilevare e correggere

Un codice con distanza minima $d_{\min}$ può:

- **rilevare** fino a $d_{\min} - 1$ errori;
- **correggere** fino a $\left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$ errori.

L'intuizione è la seguente: per correggere errori, ogni parola corrotta deve essere più vicina alla parola originale che a qualunque altra parola valida. Questo richiede "sfere" di raggio t attorno a ciascuna parola valida che non si sovrappongano, il che è possibile solo se $d_{\min} \geq 2t + 1$.

16.4. Esempi semplici: dai bit di parità ai codici

Prima di presentare il codice di Hamming, introduciamo lo strumento costruttivo fondamentale: la somma modulo 2.

Definizione 16.2— Somma modulo 2 (XOR)

Dati due bit $a, b \in \{0, 1\}$, la loro *somma modulo 2* — indicata con $a \oplus b$ e chiamata anche operazione *XOR* (dall'inglese *exclusive or*) — restituisce 0 se i due bit sono uguali e 1 se sono diversi:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

Una proprietà utile: $a \oplus b \oplus b = a$, il che rende la somma modulo 2 facilmente invertibile.

Esempio 16.1— Il codice $(2, 1)$: rilevazione senza correzione

Il codice $(2, 1)$ trasmette ogni bit due volte: le parole valide sono solo `00` e `11`, con $d_{\min} = 2$. Se Bruno riceve `01` o `10`, sa che è avvenuto un errore. Ma non può correggerlo: `01` è equidistante da `00` e `11`. Il codice *rileva* un errore singolo ma non lo *corregge*.

Esempio 16.2— Il codice $(3, 2)$: il primo bit di parità

Il codice $(3, 2)$ codifica 2 bit informativi s_1, s_2 aggiungendo un *bit di parità* $t_3 = s_1 \oplus s_2$: il risultato è 0 se $s_1 = s_2$, è 1 se sono diversi. Le quattro parole valide sono:

$s_1 s_2$	parola $t_1 t_2 t_3$
00	000
01	011
10	101
11	110

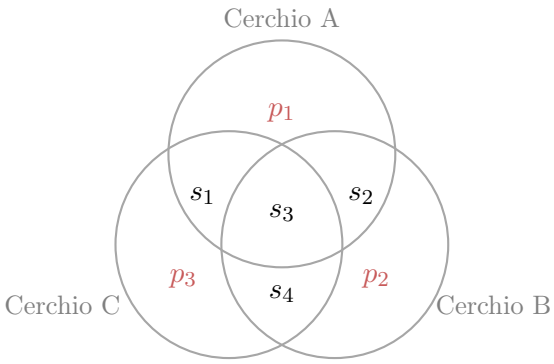
La distanza minima è $d_{\min} = 2$: errori singoli sono rilevabili, ma non correggibili. Per correggere occorre $d_{\min} \geq 3$.

17. Il codice di Hamming $(7, 4)$

Il codice di Hamming $(7, 4)$, pubblicato da Richard Hamming nel 1950, è il primo esempio di codice che corregge un errore singolo con una quantità minima di ridondanza. La storia di come nacque è emblematica. Hamming lavorava ai Bell Labs alla fine degli anni '40, quando i computer a valvole erano macchine ingombranti e inaffidabili. Il sistema di calcolo che usava era disponibile solo nei weekend e di notte; quando un errore si verificava durante un calcolo — cosa frequente — la macchina si bloccava e tutto il lavoro andava perduto. Esasperato, Hamming si pose una domanda che sembra ovvia ma che all'epoca non aveva risposta: non si può fare in modo che la macchina rilevi e corregga automaticamente i propri errori? La risposta, pubblicata nel 1950, aprì un campo di ricerca che ancora oggi è alla base di ogni sistema di comunicazione e memoria digitale.

17.1. Il diagramma dei tre cerchi

Il codice $(7, 4)$ trasmette 4 bit informativi in 7 bit totali, aggiungendo 3 bit di parità. Il modo più intuitivo di visualizzarlo è attraverso tre cerchi sovrapposti. I 7 bit vengono disposti nelle 7 regioni distinte:



I 4 bit informativi s_1, s_2, s_3, s_4 occupano le regioni di intersezione (in comune tra due o tre cerchi); i 3 bit di parità p_1, p_2, p_3 occupano le regioni esterne, ciascuno in un solo cerchio. Ogni cerchio controlla un gruppo di bit: **A** controlla $\{s_1, s_2, s_3, p_1\}$, **B** controlla $\{s_2, s_3, s_4, p_2\}$, **C** controlla $\{s_1, s_3, s_4, p_3\}$.

17.2. Costruzione dei bit di parità

Ogni bit di parità viene scelto in modo che la somma modulo 2 di tutti i bit nel suo cerchio sia zero. Questo si ottiene con le equazioni:

$$p_1 = s_1 \oplus s_2 \oplus s_3, \qquad p_2 = s_2 \oplus s_3 \oplus s_4, \qquad p_3 = s_1 \oplus s_3 \oplus s_4.$$

La scelta del segno ($p_i = \ldots$) garantisce che ogni cerchio contenga un numero pari di "uni": si parla di parità pari. Qualunque coppia di parole di codice valide differisce in almeno 3 posizioni, dunque $d_{\min} = 3$ e il codice corregge un errore singolo.

Esempio 17.1— Codifica di 1011

Vogliamo trasmettere $s_1 s_2 s_3 s_4 = \texttt{1011}$. I bit di parità sono:

$$p_1 = 1 \oplus 0 \oplus 1 = 0, \quad p_2 = 0 \oplus 1 \oplus 1 = 0, \quad p_3 = 1 \oplus 1 \oplus 1 = 1.$$

La parola trasmessa è $s_1 s_2 s_3 s_4 p_1 p_2 p_3 = \texttt{1011 001}$.

17.3. La sindrome: la firma di ogni errore

Bruno riceve 7 bit e ricalcola le tre somme di parità. Il vettore a tre bit risultante si chiama **sindrome**. Se nessun errore è avvenuto, tutte e tre le somme valgono 0: la sindrome è $(0, 0, 0)$. Se si è verificato un errore nel bit b, i soli cerchi che contengono b risultano violati (somma pari a 1), e il pattern di violazioni "firma" univocamente quale bit è errato:

Bit errato	Cerchio A	Cerchio B	Cerchio C	Sindrome
nessuno	0	0	0	$(0, 0, 0)$ — nessun errore
s_1	1	0	1	$(1, 0, 1)$
s_2	1	1	0	$(1, 1, 0)$
s_3	1	1	1	$(1, 1, 1)$
s_4	0	1	1	$(0, 1, 1)$
p_1	1	0	0	$(1, 0, 0)$
p_2	0	1	0	$(0, 1, 0)$
p_3	0	0	1	$(0, 0, 1)$

Le 8 sindromi distinte corrispondono ai 8 casi possibili: nessun errore più 7 posizioni di errore. La firma è univoca perché ogni bit appartiene a un sottoinsieme distinto di cerchi: la tabella delle appartenenze non ha due righe uguali. Questa non è una coincidenza: il codice è stato progettato apposta per garantire questa proprietà.

17.4. Un esempio completo di codifica, trasmissione e correzione

Esempio 17.2— Dall'errore alla correzione

Anna trasmette $s_1 s_2 s_3 s_4 = \texttt{1101}$. I bit di parità sono $p_1 = 0, p_2 = 0, p_3 = 0$, e la parola trasmessa è `1101 000`.
Il canale inverte il terzo bit: Bruno riceve $r = \texttt{1111 000}$.
Bruno ricalcola le tre somme di parità sul messaggio ricevuto:

$$\begin{aligned} \text{A} &: 1 \oplus 1 \oplus \mathbf{1} \oplus 0 = 1 \quad \text{(violata!)} \\ \text{B} &: 1 \oplus \mathbf{1} \oplus 1 \oplus 0 = 1 \quad \text{(violata!)} \\ \text{C} &: 1 \oplus \mathbf{1} \oplus 1 \oplus 0 = 1 \quad \text{(violata!)} \end{aligned}$$

La sindrome è $(1, 1, 1)$, che dalla tabella corrisponde al bit s_3. Bruno inverte il terzo bit e ottiene `1101 000`, ricostruendo correttamente il messaggio originale.

17.5. Efficienza e generalizzazione

Confrontiamo le strategie per correggere un errore singolo:

	Ripetizione 3	Codice $(3, 2)$	Hamming $(7, 4)$
Bit informativi (k)	1	2	4
Bit totali (n)	3	3	7
Bit di ridondanza	2	1	3
Tasso $R = k/n$	$1/3$	$2/3$	$4/7 \approx 57\%$
Capacità	corregge 1	rileva solo	corregge 1

Hamming $(7, 4)$ corregge errori singoli usando solo 3 bit di ridondanza per 4 bit informativi: il 57% dei bit porta informazione reale, contro il 33% della ripetizione semplice. E il vantaggio cresce con la dimensione: con r bit di parità è possibile proteggere fino a $2^r - r - 1$ bit informativi. Tre bit di parità coprono 4 bit informativi; 4 ne coprono 11; 5 ne coprono 26. La ridondanza necessaria cresce molto più lentamente dei bit protetti: un'efficienza crescente che è alla base dell'importanza pratica dei codici di Hamming, ancora ampiamente utilizzati nella memoria RAM e nei sistemi di storage.

18. Conclusioni

In queste note abbiamo percorso un cammino che parte da una riflessione generale sull'informatica e arriva ai fondamenti della teoria della codifica, seguendo il filo conduttore del legame tra informazione, incertezza e probabilità.

Nella prima parte, i concetti chiave sono i seguenti. L'**informazione** di un evento è una grandezza misurabile in bit, pari a $-\log_2 p$ dove p è la probabilità dell'evento: più un evento è raro, più ci sorprende quando accade. L'**entropia** è la media pesata dell'informazione sui possibili esiti di una variabile casuale e costituisce la misura dell'incertezza di una distribuzione. L'entropia è massima quando tutti gli esiti sono equiprobabili e si annulla quando il risultato è certo. I tre puzzle — le dodici

palline, l'indovina-numero, l'ordinamento per confronti — hanno mostrato come questi concetti abbiano un valore pratico immediato: la strategia ottimale in ciascun caso è quella che massimizza l'informazione acquisita ad ogni passo, bilanciando gli esiti il più possibile, e il limite inferiore al numero di operazioni necessarie è sempre fissato dall'entropia.

L'analisi del linguaggio naturale ha mostrato come la ridondanza dell'italiano sia descrivibile in termini di entropia condizionata: la lingua è molto lontana dall'essere casuale, e questa struttura è alla base dei moderni modelli del linguaggio. Questi ultimi apprendono la distribuzione condizionata del prossimo token attraverso reti neurali con miliardi di parametri, evitando la impossibilità computazionale di una tabella esaustiva e minimizzando, durante l'addestramento, la cross-entropia tra il modello e il testo osservato.

Nella seconda parte, l'entropia ha rivelato il suo ruolo di *limite universale* nella teoria della codifica. Il **teorema di Shannon sulla codifica di sorgente** afferma che nessun codice può comprimere al di sotto di $H(X)$ bit per simbolo; la **codifica di Huffman** è la risposta ottimale a questo problema tra tutti i codici prefisso. Sul lato della protezione dagli errori, la distanza di Hamming e il **codice** $(7, 4)$ di Hamming mostrano come la ridondanza strutturata permetta di correggere errori con un overhead molto inferiore alla semplice ripetizione.

Il quadro finale è unitario e potente: compressione e correzione degli errori sono due obiettivi apparentemente opposti, ma entrambi governati dagli stessi principi della teoria dell'informazione fondata da Shannon nel 1948. Ancora oggi, ogni file ZIP che comprimiamo, ogni fotografia JPEG che condividiamo, ogni pacchetto di dati che viaggia in rete porta con sé, invisibile ma presente, il contributo di Shannon, Huffman, Hamming e degli altri pionieri della teoria dell'informazione.